Labo 15 — Labo pratique : une base qui survit à ses Pods

Objectif : demander un volume et voir le cluster le fabriquer, écrire dedans depuis deux Pods successifs, puis déployer PostgreSQL en StatefulSet avec son Service headless, supprimer le Pod, le StatefulSet, revenir à zéro réplica — et retrouver les données à chaque fois.

Prérequis — Labo 14 terminé : k3s ; l'image `postgres:16-alpine` sera tirée par le cluster depuis Docker Hub.

Fichiers fournis (`files/`) - `pvc-demo.yaml` — une PVC de 100 Mi et un Pod `writer` qui écrit dedans. - `secret-db.yaml` — le mot de passe de la base (labo 14). - `statefulset-db.yaml` — le Service headless `db` et le StatefulSet PostgreSQL avec `volumeClaimTemplates`.

Étape 1 — La StorageClass et une première PVC

```
mkdir -p ~/labo-docker/15 && cd ~/labo-docker/15 && cp <chemin-du-labo>/files/* .
kubectl create namespace lab15 ; kubectl config set-context --current --namespace=lab15
kubectl get storageclass
kubectl apply -f pvc-demo.yaml
sleep 3 ; kubectl get pvc
sleep 10 ; kubectl get pvc ; kubectl get pv
```

Observez `local-path (default) rancher.io/local-path Delete WaitForFirstConsumer` ; une PVC `demo-data` `Pending` si vous êtes rapide, puis `Bound` à un volume `pvc-<uuid>` de `100Mi RWO` ; et un PV du même nom, `RECLAIM POLICY Delete`, `CLAIM lab15/demo-data`.

```
PV=$(kubectl get pvc demo-data -o jsonpath='{.spec.volumeName}')
kubectl get pv $PV -o jsonpath='{.spec.local.path}{"\n"}
{.spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values[0]}{"\n"}'
kubectl exec writer -- cat /data/journal.txt
sudo ls /var/lib/rancher/k3s/storage/
```

Observez le chemin `/var/lib/rancher/k3s/storage/pvc-<uuid>_lab15_demo-data`, l'affinité vers votre nœud, une ligne `ecrit par writer le …` dans le fichier, et le répertoire du volume sur le disque du nœud.

Explication. Vous avez demandé 100 Mi ; la StorageClass a attendu le premier Pod (`WaitForFirstConsumer`), puis créé un répertoire sur le nœud et un PV lié à votre PVC. Le Pod ne connaît que la PVC.

Étape 2 — Le volume survit au Pod, pas à la PVC

```
kubectl delete pod writer ; kubectl apply -f pvc-demo.yaml ; sleep 8
kubectl exec writer -- cat /data/journal.txt
kubectl delete pod writer ; kubectl delete pvc demo-data ; sleep 5 ; kubectl get pv | grep -c
demo-data
```

Observez `persistentvolumeclaim/demo-data unchanged` et un nouveau `writer` qui lit **deux** lignes : la sienne et celle du Pod précédent ; puis, après suppression de la PVC, `0` : le PV (politique `Delete`) a disparu avec les données.

Explication. La PVC tient le volume ; les Pods passent. Supprimer la PVC déclenche la politique de récupération.

Étape 3 — PostgreSQL en StatefulSet

```
kubectl apply -f secret-db.yaml -f statefulset-db.yaml
kubectl rollout status statefulset/db --timeout=120s
kubectl get statefulset,pods,pvc,svc
```

Observez (un avertissement `spec.SessionAffinity is ignored for headless services` est sans conséquence) : `statefulset.apps/db 1/1`, un Pod nommé **`db-0`** — pas de hash —, une PVC **`data-db-0`** de `500Mi` liée, et un Service `db` de `CLUSTER-IP None`.

```
kubectl run client --image=docker.io/library/alpine --restart=Never --command -- sleep 600 ;
sleep 6
kubectl exec client -- getent hosts db db-0.db
kubectl exec client -- sh -c 'nc -zv db-0.db.lab15.svc.cluster.local 5432'
kubectl delete pod client --now
```

Observez la même IP sous deux noms, `db.lab15.svc.cluster.local` et **`db-0.db.lab15.svc.cluster.local`**, et le port 5432 ouvert : le Service headless donne un nom à chaque Pod.

Explication. `volumeClaimTemplates` a créé `data-db-0` pour `db-0` ; le Service `clusterIP: None` ne répartit pas, il nomme.

Étape 4 — Casser le Pod, garder la base

```
kubectl exec db-0 -- psql -U postgres -c "CREATE TABLE clients(id serial, nom text); INSERT
INTO clients(nom) VALUES ('Durand'),('Peeters');"
kubectl delete pod db-0 ; sleep 2 ; kubectl get pods
kubectl rollout status statefulset/db --timeout=120s
kubectl get pvc
kubectl exec db-0 -- psql -U postgres -c "SELECT * FROM clients;"
```

Observez `CREATE TABLE` / `INSERT 0 2` ; après la suppression, un Pod qui s'appelle **de nouveau `db-0`** (`AGE 2s`) ; la même PVC `data-db-0` ; et les deux clients retrouvés.

```
kubectl scale statefulset db --replicas=0 ; sleep 10 ; kubectl get pods ; kubectl get pvc
kubectl scale statefulset db --replicas=1 ; kubectl rollout status statefulset/db --
timeout=120s
kubectl exec db-0 -- psql -U postgres -c "SELECT count(*) FROM clients;"
```

Observez `No resources found` pour les Pods mais `data-db-0` toujours `Bound` à zéro réplica, et `count = 2` au retour.

Explication. Le StatefulSet rattache le Pod `db-0` à la PVC `data-db-0` à chaque recréation ; réduire à zéro ne libère pas le volume.

Étape 5 — Sauvegarder, supprimer le StatefulSet, recréer

```
kubectl exec db-0 -- pg_dump -U postgres > dump.sql ; grep -A3 'COPY public.clients' dump.sql
kubectl delete statefulset db ; sleep 8 ; kubectl get pods ; kubectl get pvc
kubectl apply -f statefulset-db.yaml ; kubectl rollout status statefulset/db --timeout=120s
kubectl exec db-0 -- psql -U postgres -c "SELECT count(*) FROM clients;"
kubectl get pv
```

Observez le `dump.sql` avec les deux lignes (sauvegarde logique, à chaud, comme au labo 06) ; après `delete statefulset`, plus de Pod mais `data-db-0` **toujours là** ; après recréation, `count = 2` ; et un PV `Bound` à `lab15/data-db-0`.

Explication. Supprimer un StatefulSet ne supprime jamais ses PVC : la base attend le prochain StatefulSet du même nom.

Nettoyage

```
kubectl delete namespace lab15 ; sleep 20
kubectl get pv | grep -c lab15
kubectl config set-context --current --namespace=default
```

Observez `0` : la suppression du namespace a supprimé la PVC, donc le PV et les données (politique `Delete`). C'est le seul geste de ce labo qui détruit réellement la base. Gardez `dump.sql` : c'est votre sauvegarde.

Ce que vous devez pouvoir affirmer maintenant

- `local-path`, `WaitForFirstConsumer`, `Delete` : une PVC `Pending` jusqu'au premier Pod, un PV `pvc-<uuid>` dans `/var/lib/rancher/k3s/storage/`, avec affinité de nœud.
- Deux Pods successifs lisent le même `journal.txt` ; supprimer la PVC supprime le PV.
- StatefulSet : `db-0`, `data-db-0`, Service `CLUSTER-IP None`, `db-0.db.lab15.svc.cluster.local`.
- Pod supprimé, scale à 0, StatefulSet supprimé : `data-db-0` reste, `count = 2` à chaque retour.
- Namespace supprimé : PV et données disparus ; `pg_dump` par `kubectl exec` est la sauvegarde.